VERIQTA

Your First AI Agent on AWS

Build One Step by Step



PRACTICAL EXAMPLES | REAL AWS SETUP | STEP-BY-STEP GUIDE | TROUBLESHOOTING TIPS | BEST PRACTICES

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Your First AI Agent on AWS: Build One Step by Step

1. What Is an AI Agent?

- An AI agent is an AI system that can reason, make decisions, and use tools to complete tasks for you.
- It can understand a goal, break it down into steps, call AWS services, and give you the result.

2. AI Agent vs Chatbot

Feature	AI Agent	Chatbot
Purpose	Completes tasks	Answers questions
Reasoning	Yes	Limited
Uses tools	Yes (AWS services)	No or very limited
Can take action	Yes	No
Example	Create an EC2 instance for me"	Explain what EC2 is

3. What We Are Building

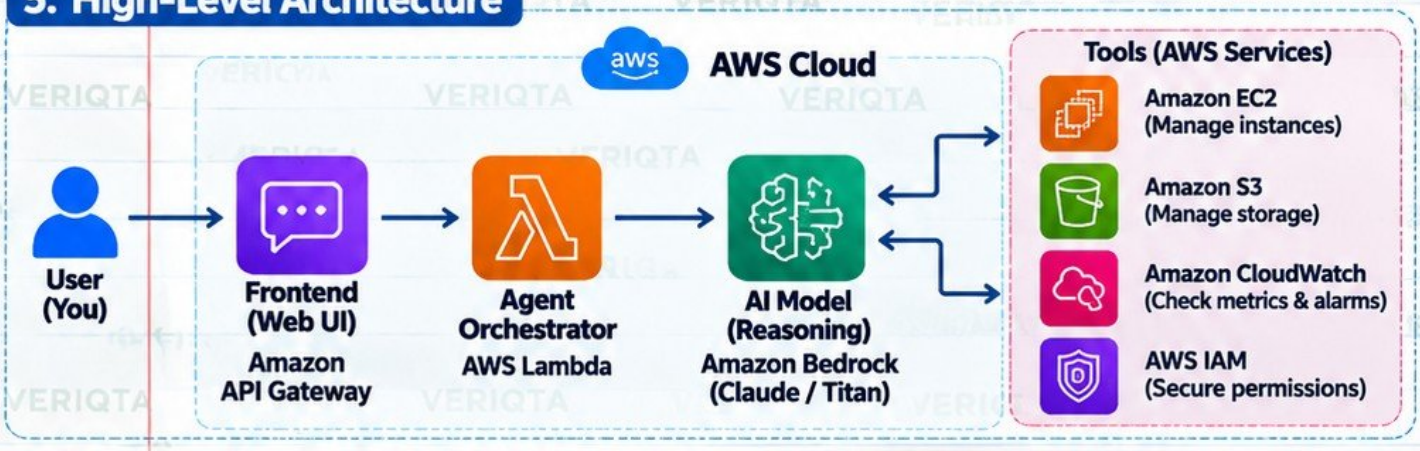
We will build a real AI agent on AWS that acts as a Cloud Operations Assistant. You can ask it to perform infrastructure tasks, and it will use AWS services to complete them.

4. Example Requests

- Create an EC2 instance
- Check the status of my instances
- List S3 buckets
- Create an S3 bucket
- Show recent CloudWatch alarms



5. High-Level Architecture



6. AWS Services We Will Use



7. What You Will Learn

- How to design a real AI agent on AWS
- How it uses tools to perform tasks
- How to secure it with IAM
- How to test and improve it
- How to run it in a production-like setup

8. Prerequisites

- AWS account (Free Tier is enough)
- Basic understanding of AWS
- Basic Python knowledge (helpful but not mandatory)
- A code editor (VS Code)

9. Milestone for This Page

Understand the complete system before building it in the next pages.

Your First AI Agent on AWS: Build One Step by Step

02 Create Your AWS Account and Set a Budget

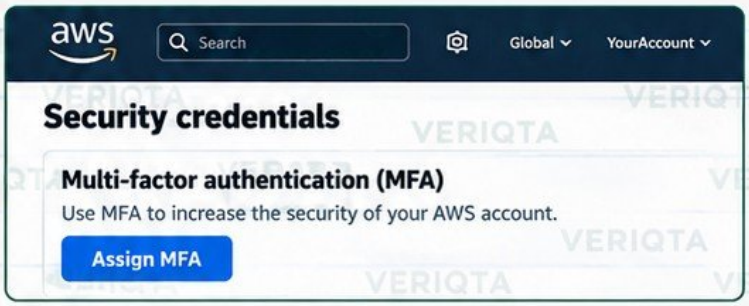
1 Create an AWS Account

1. Go to https://aws.amazon.com
2. Click Create an AWS Account
3. Enter your email and account name
4. Provide payment method (credit or debit card)
5. Complete identity verification
6. Sign in to the AWS Management Console



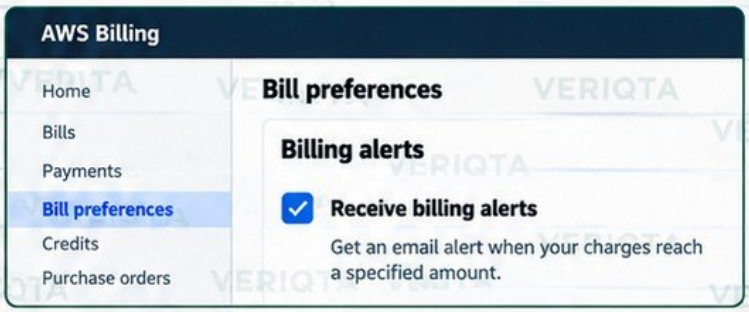
2 Enable Multi-Factor Authentication (MFA)

1. In the AWS console, click your account name
2. Choose Security credentials
3. Under Multi-factor authentication (MFA), click Assign MFA
4. Use an authenticator app (e.g. Google Authenticator)
5. Scan the QR code and enter the verification code



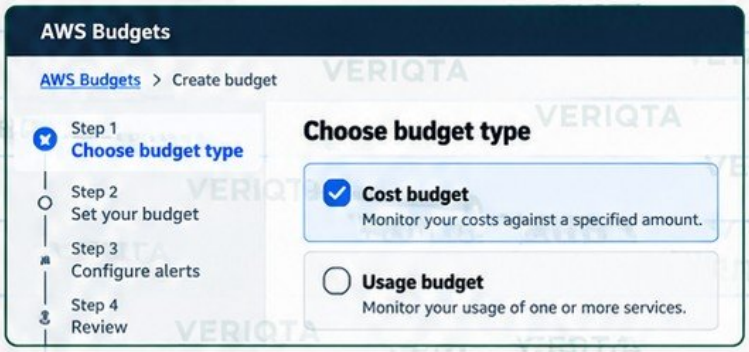
3 Set Up Billing Alerts (SNS)

1. Go to Amazon SNS
2. Click Create topic
3. Choose Email and enter your email
4. Confirm the subscription from your email
5. Go to AWS Billing > Bill preferences
6. Enable Receive billing alerts



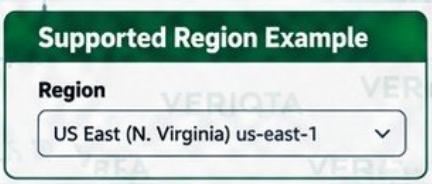
4 Create an AWS Budget

1. Go to AWS Budgets
2. Click Create budget
3. Choose Cost budget
4. Set a monthly budget (e.g. $10 or $20)
5. Configure alert thresholds (e.g. 50%, 80%, 100%)
6. Add your email for notifications
7. Review and create the budget



5 Choose a Supported Region

1. Use a region that supports the services we will use (e.g. us-east-1, us-west-2 or eu-west-1).
2. Keep the same region for all resources in this project.
3. Check service availability in the AWS documentation.



Tip

Use us-east-1 for this handbook to avoid missing service features.

Key Points

- Use a real email and keep your account secure.
- Always enable MFA.
- Set a budget to avoid unexpected charges.
- Use a supported region for all services.

Milestone

AWS account ready with cost controls configured.

Install Python, AWS CLI, and Boto3

Install the required tools, create a Python virtual environment, configure temporary AWS credentials, and verify with aws sts get-caller-identity.

1 Install Python

Check if Python is installed:

```
$ python3 --version
```

Expected output:

```
Python 3.11.6
```

If not installed (Ubuntu/Debian):

```
$ sudo apt update
$ sudo apt install -y python3 python3-venv
```

2 Install AWS CLI

Install the latest AWS CLI v2 (Linux):

```
$ curl "https://awscli.amazonaws.com/
  awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
$ unzip awscliv2.zip
$ sudo ./aws/install
```

Verify installation:

```
$ aws --version
```

Expected output:

```
aws-cli/2.15.0 Python/3.11.6 Linux/5.15.0-1051-aws
exe/x86_64.ubuntu.22
```

3 Create a Python Virtual Environment

Create and activate a virtual environment for this project:

```
$ python3 -m venv ai-agent-env
$ source ai-agent-env/bin/activate
```

Your terminal prompt should show (ai-agent-env):

```
(ai-agent-env) $
```

Why use a virtual environment?

- Keeps project dependencies isolated
- Avoids conflicts with system packages
- Makes the project easy to reproduce
- Good practice for real-world projects

4 Install Boto3 (AWS SDK for Python)

With the virtual environment activated, install Boto3:

```
(ai-agent-env) $ pip install boto3
```

Verify installation:

```
(ai-agent-env) $ pip show boto3
```

5 Configure AWS Credentials

Configure temporary AWS credentials using the AWS CLI:

```
$ aws configure
```

You will be prompted for:

AWS Access Key ID:	(your access key)
AWS Secret Access Key:	(your secret key)
Default region name:	(e.g. us-east-1)
Default output format:	(e.g. json)

6 Verify the Configuration

Check that your credentials are working:

```
$ aws sts get-caller-identity
```

Expected output (example):

```
{
  "UserId": "AIDAEXAMPLE1234567890",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/your-user-name"
}
```

Common Issues

- **aws: command not found**
  → AWS CLI not installed or not in PATH.
- **AccessDenied**
  → Incorrect or expired credentials.
- **InvalidClientTokenId**
  → Wrong access key or secret key.
- **NoRegionError**
  → Set a valid AWS region with aws configure.

Milestone

Working local development environment.

Next Step

Start building your first AI agent on AWS using Python and Boto3.

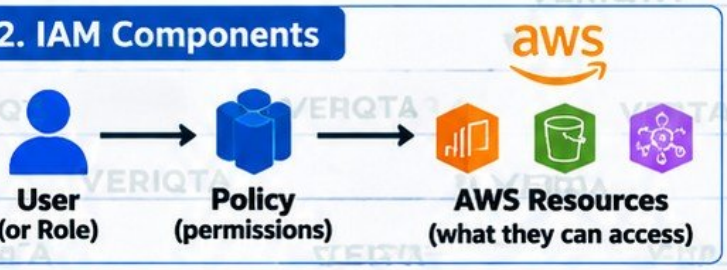
Configure IAM Permissions Safely

Create a least-privilege development identity, understand AWS roles and policies, and configure the read-only permissions needed by the project.

1. What is IAM?

- IAM (Identity and Access Management) controls who can access AWS and what they can do.
- You use IAM users, groups, roles and policies to apply permissions.

2. IAM Components



3. Principle of Least Privilege

- Give only the permissions needed.
- Start with read-only access.
- Add more permissions only when required.
- Avoid using the root account.

4. Create an IAM User for Development

1. Sign in to AWS Management Console.
2. Go to IAM → Users → Create user.
3. Enter a username (e.g. ai-agent-dev).
4. Select "Provide user access to the AWS Management Console" or programmatic access.
5. Set a strong password and enable MFA.

5. Attach a Read-Only Policy (Quick Option)

1. Go to Permissions → Attach policies directly.
2. Search for and select AWSReadOnlyAccess.
3. Click Next and create the user.

6. Custom Read-Only Policy (Recommended)

Example: allow read-only access to key services

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:Describe*",
        "s3:List*",
        "s3:Get*",
        "iam:List*",
        "cloudwatch:Describe*"
      ],
      "Resource": "*"
    }
  ]
}
```

7. Enable MFA (Recommended)

- In the IAM user details, go to Security credentials.
- Choose Multi-factor authentication (MFA).
- Use an authenticator app (e.g. Google Authenticator).
- Scan the QR code and enter the 6-digit code.

8. Verify the User

- Sign out and sign in using the new IAM user.
- Confirm you can access the console.
- Try listing S3 buckets (should work with read-only access).

```
aws s3 ls
# Lists your S3 buckets (if allowed)
```

Important Security Notes

- Never use the root account for daily work.
- Do not give AdministratorAccess for this project.
- Use least-privilege permissions.
- Use MFA for all human users.
- Avoid hardcoding AWS credentials.
- Use IAM roles when running on AWS services.

Milestone: Secure AWS access without hardcoded credentials.

Enable Amazon Bedrock and Select an AI Model

1. What is Amazon Bedrock?

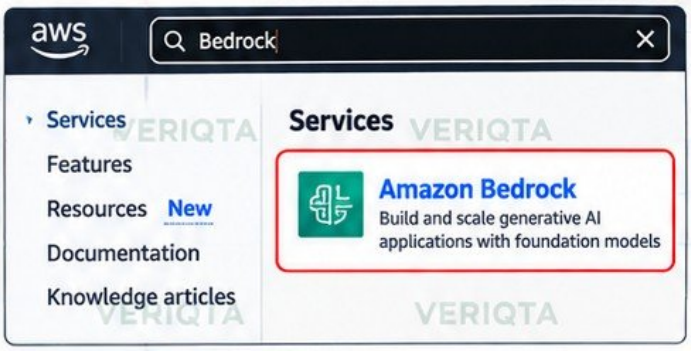
- Amazon Bedrock is a **fully managed service** that gives you access to powerful **foundation models (FMs)** from companies like Amazon, Anthropic, Meta, and others.
- You can build AI applications without managing infrastructure.
- You can use models for **text, images, and tool use** (function calling, agents).

2. Why Use Bedrock?

- Access to top AI models (Claude, Llama,
- Amazon Titan, etc.)
- No model hosting or GPU management
- Pay only for what you use
- Built-in security, privacy, and compliance
- Supports Agents and tool use for real-world applications

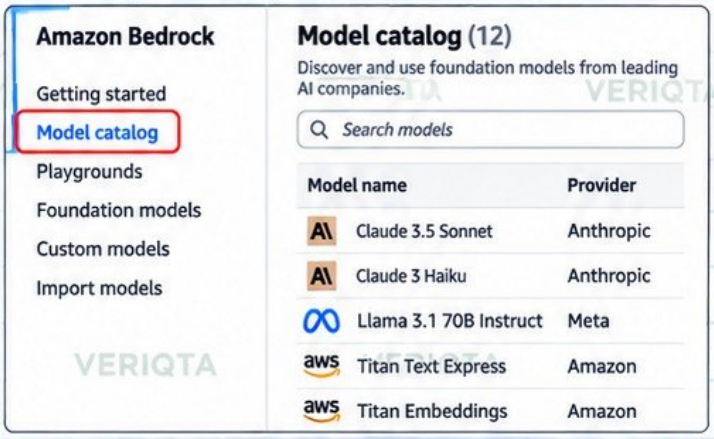
3. Enable Amazon Bedrock

1. Sign in to the AWS Management Console.
2. Search for **Bedrock** and open **Amazon Bedrock**.



4. Explore the Model Catalog

1. In the left menu, click **Model catalog**.
2. Browse available models from Amazon, Anthropic, Meta, and others.
3. Check model details, capabilities, and supported Regions.



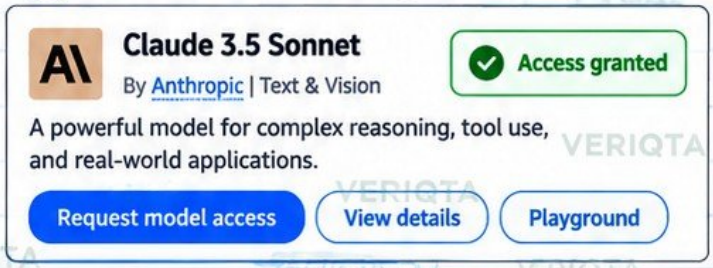
5. Check Model Availability in Your Region

1. At the top right, check your AWS Region.
2. Select a supported Region (e.g. us-east-1, us-west-2, eu-west-1).
3. Not all models are available in every Region.



6. Request Model Access (If Required)

1. Click on a model (e.g. **Claude 3.5 Sonnet**).
2. If access is required, click **Request model access**.
3. Wait for the status to change to **Access granted**.



7. Understand Token Costs

- Each model has **input and output token pricing.**
- Check the **pricing section** on the model page.
- Costs vary by model and Region.
- Start with **smaller, cheaper models** while testing.

Milestone
A foundation model is accessible in the selected AWS Region.

Create Your AI Agent Project Structure

Build the project directory, Python modules, configuration files, .gitignore, dependencies, and a secure environment configuration.

1. Create the Project Directory

```
# Create project folder
mkdir aws-ai-agent
cd aws-ai-agent
# Create initial structure
mkdir -p src/{agent,tools,utils} \
config tests data
```

What this does:

- Creates the main project folder.
- Adds subdirectories for code, configuration, tests and sample data.
- Keeps the project organized from the start.

2. Project Structure

```
aws-ai-agent/
├── src/
│   ├── agent/
│   │   ├── __init__.py
│   │   └── agent.py
│   ├── tools/
│   │   ├── __init__.py
│   │   └── tools.py
│   └── utils/
│       ├── __init__.py
│       └── helpers.py
├── config/
│   └── settings.py
├── tests/
│   └── test_agent.py
├── data/
│   └── sample_data.json
├── .env
├── .env.example
├── requirements.txt
├── .gitignore
└── README.md
```

3. Example Python Module

src/agent/agent.py

```
import os

def run_agent():
    print("AI Agent is ready to run!")

if __name__ == "__main__":
    run_agent()
```

4. Configuration File

config/settings.py

```
import os

AWS_REGION = os.getenv("AWS_REGION", "us-east-1")
MODEL_ID = os.getenv("MODEL_ID", "amazon.titan-tg1-large")
LOG_LEVEL = "INFO"
```

5. Environment Variables

.env.example

```
# AWS Configuration
AWS_REGION=us-east-1
MODEL_ID=amazon.titan-tg1-large
# Additional Settings
LOG_LEVEL=INFO
APP_NAME=aws-ai-agent
```

6. Dependencies

requirements.txt

```
boto3==1.34.0
langchain==0.3.0
langchain-aws==0.2.0
python-dotenv==1.0.1
pydantic==2.7.0
```

7. .gitignore

.gitignore

```
# Python
__pycache__/
*.pyc
.env
.env.*
.venv/
venv/  # AWS
```

8. Initialize Git Repository

```
git init
git add .
git commit -m "Initial commit"
```

Security Reminder

- Never commit your real .env file.
- Keep AWS credentials out of code.
- Use .env and add it to .gitignore.

✓ **Milestone** Reproducible project ready for development.

VERIQTA

Make Your First Real AI Model API Call

Page 07

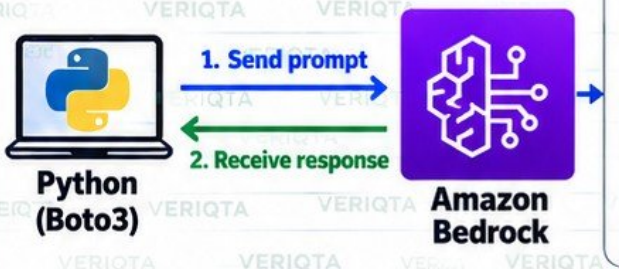
Use Boto3 and the Amazon Bedrock Converse API to send a prompt, receive a response, and inspect token usage and errors.

1. What We Are Doing

We will use Python (Boto3) to call **Amazon Bedrock** and send a prompt to a real AI model using the **Converse API**.

The model sends back a response, and we will also check token usage and handle errors.

2. How It Works



3. Prerequisites

- AWS account with access to Amazon Bedrock
- Bedrock model access enabled (e.g. Claude 3)
- IAM user or role with Bedrock permissions
- Python 3.9+ installed
- Boto3 library installed
- AWS CLI configured (or IAM role if using EC2)

4. Install and Configure Boto3

```
# Install Boto3
pip install boto3 --upgrade

# Ronfigure AWS credentials
(if not already done)
aws configure
```

You need:

- AWS Access Key ID
- AWS Secret Access Key
- Default Region (e.g. us-east-1)

5. Python Code: Call Amazon Bedrock Converse API

```
import boto3
import json

# Create Bedrock client
bedrock = boto3.client("bedrock-runtime", region_name="us-east-1")

# Define the model ID (example: Claude 3 Sonnet)
model_id = "anthropic.claude-3-sonnet-20240229-v1:0"

# Create the request
response = bedrock.converse(
    modelId=model_id,
    messages=[
        {
            "role": "user",
            "content": [{"text": "Explain DevOps in simple terms."}]
        }
    ],
    inferenceConfig={
        "maxTokens": 500,
        "temperature": 0.7
    }
)

# Extract and print the response
print("Response:")
print(response['output']['message']['content'][0]['text'])

# Print token usage
print("\nToken Usage:")
print(json.dumps(response.get('usage', {}), indent=2))
```

6. Example Output

```
Response:
DevOps is a way of working that
combines software development (Dev)
and IT operations (Ops) to deliver
applications faster, more reliably,
and with continuous improvement.

Token Usage:
{
   "inputTokens": 15,
   "outputTokens": 82,
   "totalTokens": 97
}
```

7. Common Errors and Fixes

Error Message	Reason & Fix
AccessDeniedException	Bedrock access not enabled. Request model access in AWS Console.
ValidationException	Model ID is incorrect. Use a valid model ID for your Region.
ThrottlingException	Too many requests. Wait and retry with backoff.
Networking errors	Check your internet connection and AWS credentials.

8. Try This

1. Change the prompt to ask about a topic you are learning.
2. Try a different model (e.g. Titan or Meta Llama) if you have access.
3. Print only the model response without token usage.
4. Intentionally use an invalid model ID and observe the error.

9. Milestone

Python successfully communicates with an AWS-hosted AI model.

VERIQTA

Build a Multi-Turn AI Assistant

What You Will Do on This Page

- Add system instructions to guide the assistant.
- Maintain conversation history for multiple turns.
- Handle responses from Amazon Bedrock.
- Implement basic context management.
- Test the assistant with a real multi-turn conversation.

Final Result (Multi-Turn Conversation)

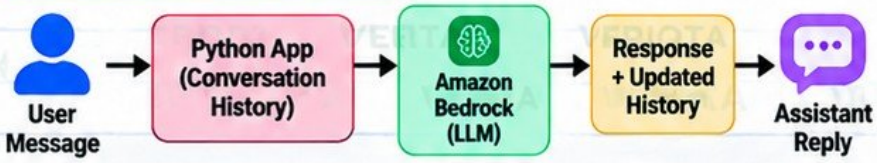
User: What is AWS?

Assistant: AWS is Amazon Web Services, a cloud platform that provides compute, storage, ...

User: Can you give an example?

Assistant: Sure! For example, you can use Amazon EC2 to run virtual servers in the cloud ...

Architecture Overview



1 Step 1: Install and Configure AWS SDK

```
# Install AWS SDK for Python (if not installed)
pip install boto3
# Configure AWS credentials (if not already configured)
aws configure
# Test connection to Bedrock
import boto3
bedrock = boto3.client("bedrock-runtime", region_name="us-east-1")
print("Bedrock client created successfully!")
```

2 Step 2: Define System Instructions

```
system_prompt = (
  """
  You are a helpful DevOps and
  cloud assistant.
  Give clear, accurate and
  practical answers.
  Keep responses concise and
  beginner-friendly.
  """)
```

3 Step 3: Maintain Conversation History

```
messages = [
    {"role": "system", "content": system_prompt}
]
def add_user_message(user_input):
    messages.append({"role": "user", "content": user_input})
def add_assistant_message(response):
    messages.append({"role": "assistant","content": response})
```

4 Step 4: Get Response from Amazon Bedrock

```
import json
def get_bedrock_response():
    response = bedrock.converse(
        modelId="anthropic.claude-3-sonnet-20240229-v1:0",
        messages=messages
    )
    reply = response["output"]["message"]["content"][0]["text"]
    add_assistant_message(reply)
    return reply
```

5 Step 5: Simple Multi-Turn Chat Loop

```
print("Type 'exit' to quit\n")
while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break
    add_user_message(user_input)
    reply = get_bedrock_response()
    print(f"\nAssistant: {reply}\n")
```

Context Management Tips

- Keep recent messages only (e.g. last 5–10 turns).
- Remove very old messages to save tokens.
- Summarize long conversations if needed.
- Avoid storing sensitive information.
- Use clear system instructions to control behaviour.

Example Multi-Turn Test

1. Ask: What is AWS?
2. Ask a follow-up: Give an example.
3. Ask: How do I create an S3 bucket?
4. Ask: Can you summarize our conversation?

Milestone: The assistant can maintain a short conversation.

Next Page Add memory using a simple file or database.

VERIQTA

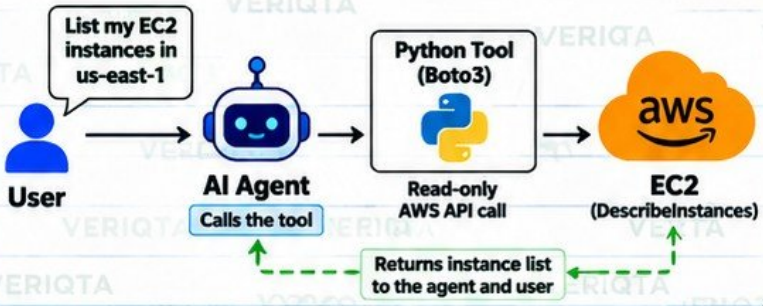
09. Give Your AI Agent Its First Tool

Create a Python function that uses Boto3 to list Amazon EC2 instances. Define the tool's input schema and restrict it to approved read-only operations.

1. What We Are Building

- A tool that lets the AI agent list your EC2 instances using AWS.
- The tool uses a Python function with Boto3 (AWS SDK for Python).
- The tool only performs read-only operations (no changes to AWS).
- We also define an input schema so the agent knows what parameters it can accept.

2. How It Works



3. IAM Permissions (Read-Only)

Attach a policy to the IAM user/role used by the agent. Allow only the describe action.

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Effect": "Allow",
         "Action": [
            "ec2:DescribeInstances"
         ],
         "Resource": "*"
      }
   ]
}
```

4. Python Tool: List EC2 Instances

```
import boto3
from typing import Optional, Dict, Any, List

def list_ec2_instances(
    region: Optional[str] = None,
    state: Optional[str] = None
) -> Dict[str, Any]:
    """List EC2 instances (read-only)."""
    ec2 = boto3.client('ec2', region_name=region)
    filters = []

    if state:
        filters.append({
            'Name': 'instance-state-name',
            'Values': [state]
        })

    response = ec2.describe_instances(Filters=filters)
    instances = []

    for reservation in response['Reservations']:
        for instance in reservation['Instances']:
            instances.append({
                'InstanceId': instance['InstanceId'],
                'State': instance['State']['Name'],
                'InstanceType': instance['InstanceType'],
                'PublicIpAddress': instance.get('PublicIpAddress','N/A'),
                'PrivateIpAddress': instance.get('PrivateIpAddress','N/A')
            })
    return {
        'count': len(instances),
        'instances': instances
    }
```

5. Tool Input Schema (for AI Agent)

```
{
   "type": "object",
   "properties": {
      "region": {
         "type": "string",
         "description": "AWS region (optional)",
         "enum": ["us-east-1", "us-west-2",
                  "eu-west-1"]
      },
      "state": {
         "type": "string",
         "description": "Instance state filter",
         "enum": ["running", "stopped", "pending",
                  "stopping", "terminated"]
      }
   },
   "additionalProperties": false
}
```

6. Example Usage

```
>>> list_ec2_instances(region="us-east-1")
>>> list_ec2_instances(region="us-east-1", state="running")
```

7. Example Output (Truncated)

```
{
   "count": 2,
   "instances": [
      {
         "InstanceId": "i-0abc123def456",
         "State": "running",
         "InstanceType": "t3.micro",
         "PublicIpAddress": "54.12.34.56",
         "PrivateIpAddress": "10.0.1.10"
      }
   ]
}
```

8. Important Notes

- This tool only uses `ec2:DescribeInstances` (read-only).
- Restrict the IAM role to the minimum permissions needed.
- You can add more describe actions later (e.g. DescribeVolumes).
- Always validate user input in production.

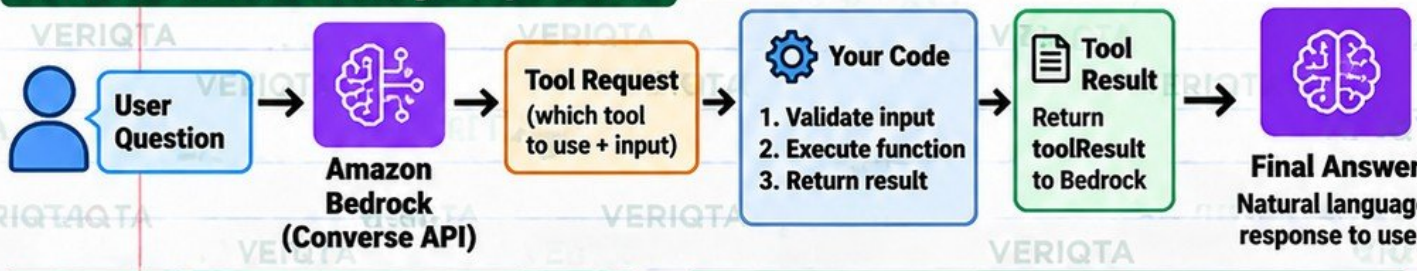
9. Milestone

A real AWS inspection tool is ready.

Page 10: Build the Agent's Tool-Calling Loop

Goal: Implement the Converse API tool request, execute the validated function, return toolResult, and generate a final natural-language answer.
Milestone: The AI agent can choose and use its first AWS tool.

1. How the Tool-Calling Loop Works



2. Define the Tool for the Agent

Example: A tool to list S3 buckets in your account.

```
tools = [
  {
    "toolSpec": {
      "name": "list_s3_buckets",
      "description": "List all S3 buckets
      in the AWS account.",
      "inputSchema": {
        "json": {
          "type": "object",
          "properties": {}
        }
      }
    }
  }
]
```

3. Call the Converse API

Send the user message and tool definition to Bedrock.

```
import boto3

bedrock = boto3.client("bedrock-runtime",
  region_name="us-east-1")

response = bedrock.converse(
  modelId="anthropic.claude-3-5-sonnet-20241022-v2:0",
  messages=[
    "role": "user",
    "content": [{"text": "List my S3 buckets"}]
  }],
  tools=tools
)
```

The model may respond with a tool request instead of a final answer. Your code must handle this, execute the tool, and send the result back.

4. Check for a Tool Request

If the model requests a tool, extract the tool name and input.

```
output = response["output"]
message = output["message"]

if message.get("toolUse"):
  tool_name = message["toolUse"]["name"]
  tool_input = message["toolUse"]["input"]
```

5. Execute the Tool

Run the requested function with validated input.

```
if tool_name == "list_s3_buckets":
  s3 = boto3.client("s3")
  result = s3.list_buckets()
  buckets = [b["Name"] for b
             in result["Buckets"]]
```

6. Return the toolResult

Send the result back to Bedrock and get the final natural answer.

```
tool_result = {
  "toolResult": {
    "content": [{"json": {
      "buckets": buckets
    }}]
  }
}
```

7. Get the Final Answer

Call Converse again with the toolResult.

8. Example Final Output

Here are your S3 buckets:

- veriqta-logs
- veriqta-data
- veriqta-backups

Important Notes

- Always validate tool input.
- Use least-privilege IAM permissions.
- Handle errors and return clear messages.
- You can add more tools for new capabilities.

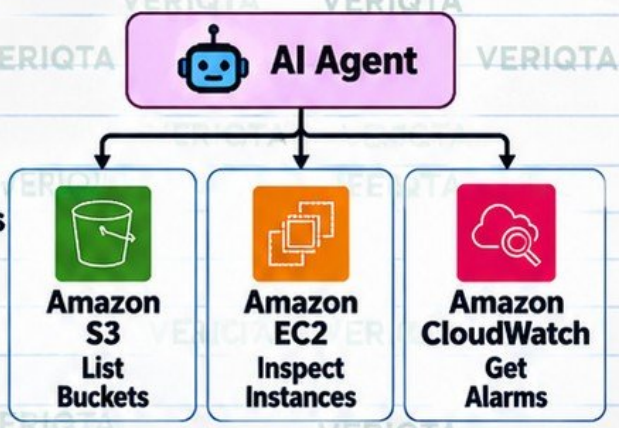
Milestone: The AI agent can choose and use its first AWS tool.

Add More AWS Tools to Your AI Agent

In this page, you will build real Boto3 functions to list S3 buckets, inspect EC2 instances and retrieve CloudWatch alarms. You will register each tool with a valid input schema, validate Regions and arguments, and test them independently.

1. Overview

- We will create three read-only AWS tools:
  - list_s3_buckets – list your S3 buckets
  - list_ec2_instances – inspect EC2 instances
  - list_cloudwatch_alarms – retrieve CloudWatch alarms
- Each tool will accept a Region and optional arguments.
- We will validate inputs to prevent errors.
- Then we will test each function before integrating with the AI agent.



2. Install and Configure Boto3

```
# Install Boto3 (if not installed)
pip install boto3

# Configure AWS credentials (one time)
aws configure
AWS Access Key ID [None]: YOUR_ACCESS_KEY
AWS Secret Access Key [None]: YOUR_SECRET_KEY
Default region name [None]: us-east-1
Default output format [None]: json
```

Prerequisites

- AWS account with IAM user or role
- Permissions: S3 Read, EC2 Read, CloudWatch Read
- Boto3 installed
- AWS credentials configured
- Choose a supported Region (e.g. us-east-1, eu-west-1)

3. Helper Functions

```
import boto3
from botocore.exceptions import ClientError, NoCredentialsError

def get_client(service_name, region):
    if not region:
        raise ValueError("Region is required")
    return boto3.client(service_name, region_name=region)

def validate_region(region):
    allowed_regions = [r['RegionName'] for r in
        boto3.client('ec2').describe_regions()["Regions']]
    if region not in allowed_regions:
        raise ValueError(f"Invalid region: {region}")
    return region
```

Region Validation

- We validate the Region to prevent invalid API calls.
- You can also hardcode a list of allowed Regions if needed.
- Using an invalid Region will cause ta ClientError.

Best Practice

- Use least-privilege IAM permissions.
- Keep your AWS credentials secure.
- These tools are read-only and safe to use for inspection.

Build a Complete Multi-Tool AI Agent

VERIQTA

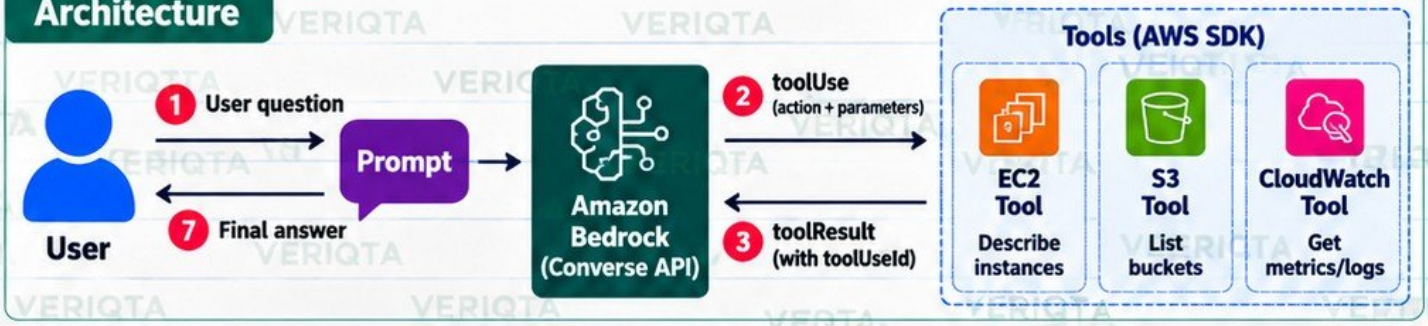
What We Are Building

- A real AI agent using Amazon Bedrock's Converse API.
- The agent can use multiple tools (EC2, S3, CloudWatch).
- It chooses the right tool based on the user's request.
- It handles `toolUse`, executes the function, returns `toolResult` with the correct `toolUseId`, and continues until a final response.
- We set a maximum tool-call limit to prevent loops.

Milestone

✅ A user can ask about EC2, S3, or CloudWatch, and the agent chooses the appropriate tool.

Architecture



Tool Definition (Send to Bedrock)

```
"tools": [
  {
    "toolSpec": {
      "name": "list_ec2_instances",
      "description": "List EC2 instances in the account",
      "inputSchema": {
        "json": {
          "type": "object",
          "properties": {
            "state": { "type": "string" }
          }
        }
      }
    }
  }
```

Handle toolUse and toolResult (Python)

```
max_tool_calls = 5
tool_calls = 0
messages = [{"role": "user", "content": user_question}]
while tool_calls < max_tool_calls:
    response = bedrock.converse(
        modelId=MODEL_ID,
        messages=messages,
        tools=tools
    )
    content = response["output"]["message"]["content"]
    if content[0].get("toolUse"):
        tool = content[0]["toolUse"]
        tool_name = tool["name"]
        tool_id = tool["toolUseId"]
        params = tool.get("input", {})
        result = execute_tool(tool_name, params)
        messages.append({
            "role": "user",
            "content": [{"toolResult":
                "toolUseId": tool_id,
                "content": [{"json": result}]
            }}]
        })
        tool_calls += 1
    else:
        final_answer = content[0]["text"]
        break
```

Tool Execution Examples (AWS SDK for Python)

EC2 Tool – List instances

```
import boto3
ec2 = boto3.client("ec2")
response = ec2.describe_instances(
    Filters=[{"Name": "instance-state-name", "Values": ["running"]}]
)
instances = response["Reservations"]
```

S3 Tool – List buckets

```
import boto3
s3 = boto3.client("s3")
response = s3.list_buckets()
buckets = [b["Name"] for b in response["Buckets"]]
```

CloudWatch Tool – Get metrics

```
import boto3
cw = boto3.client("cloudwatch")
response = cw.get_metric_statistics(
    Namespace="AWS/EC2",
    MetricName="CPUUtilization",
    StartTime=start_time,
    EndTime=end_time,
    Period=300,
    Statistics=["Average"]
)
```

Important Checks

- ✅ Match the tool name exactly as defined.
- ✅ Return `toolResult` with the correct `toolUseId`.
- ✅ Continue the loop until a final response is produced.
- ✅ Set a maximum tool-call limit (e.g., 5).
- ✅ Handle errors and return a helpful message.

13. Add Conversation Memory and Session Management

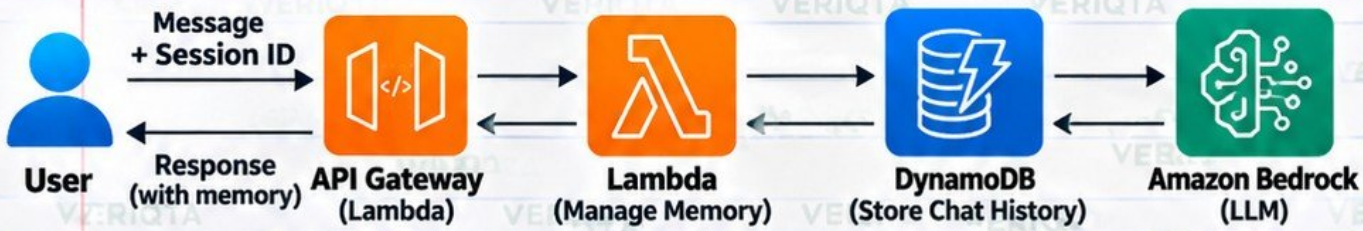
1. Why Conversation Memory?

- Users ask follow-up questions.
- The agent needs to remember previous context.
- We store short conversation history using a unique session ID.
- This improves response quality and user experience.

2. How It Works

1. User sends a message with a session ID.
2. We retrieve the previous messages for that session from DynamoDB.
3. We send the conversation history to Amazon Bedrock in the correct format.
4. We store the new message and response.
5. We limit the conversation length.
6. Each user has an isolated session.

3. Conversation Flow



4. DynamoDB Table for Chat History

Table name: ai-agent-chat-history
Partition key: session_id (String)
Sort key: timestamp (String or Number)
Attributes:

- role (user or assistant)
- content (message text)
- timestamp
- ttl (optional, for automatic deletion)

5. Example Item in DynamoDB

```
{
    "session_id": "user-123",
    "timestamp": "2026-10-24T10:15:30Z",
    "role": "user",
    "content": "List my EC2 instances",
    "ttl": 1761322530
}
```

6. Build the Message History for Bedrock

```
messages = [
    {"role": "user", "content": "List my EC2 instances"},
    {"role": "assistant", "content": "Here are your EC2
    instances..."},
    {"role": "user", "content": "Which one is running?"}
]
```

7. Limit Conversation Length

- Keep only the last 5 to 10 messages.
- Older messages can be removed or summarized.
- Use a TTL in DynamoDB to delete old sessions automatically.

8. Prevent Cross-User Data

- Always use a unique session ID per user.
- Never use a global key.
- Validate the session ID from the authenticated user.
- Use IAM and proper access control.

Milestone

The agent can answer follow-up questions about previously retrieved AWS resources.

Page 14 Handle AWS Errors, Timeouts, and Failed Tool Calls

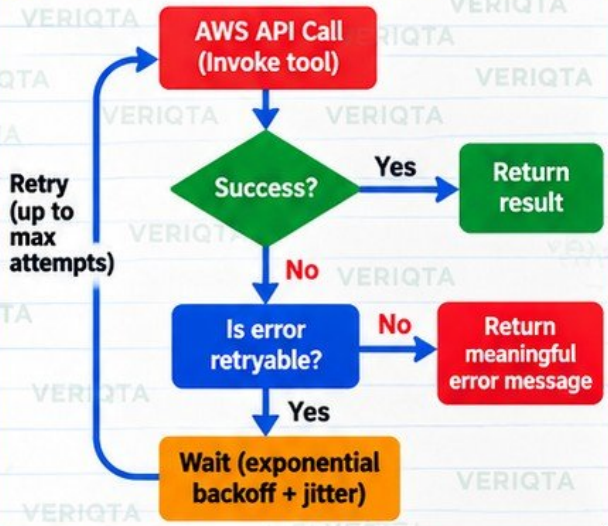
VERIQTA

Catch ClientError, handle AccessDenied, throttling, invalid Regions, missing resources, and model errors. Implement bounded retries with exponential backoff, distinguish retryable from permanent failures, and return useful error messages.

1 Common AWS Errors and What They Mean

Error	What It Means	Example Scenario
AccessDenied Exception	You don't have permission.	IAM policy does not allow the action.
ResourceNotFound Exception	The resource does not exist.	S3 bucket or Lambda function not found.
ThrottlingException	You are sending too many requests.	Too many requests to AWS API.
InvalidRegion Exception	The Region is invalid or not supported.	Using a service in the wrong Region.
ValidationException	The request is not valid.	Missing required parameter.
ServiceUnavailable Exception	AWS service is temporarily unavailable.	Try again later.
ClientError	General client-side error from boto3.	Covers many 4xx errors.

2 Retry Logic with Exponential Backoff



3 Python Example: Handle AWS Errors

```python
import boto3
from botocore.exceptions import ClientError
import time
import random

def call_aws_with_retry(client, operation, *args, **kwargs):
    max_attempts = 4
    base_delay = 1 # seconds
    for attempt in range(1, max_attempts + 1):
        try:
            response = getattr(client, operation)(*args, **kwargs)
            return response
        except ClientError as e:
            error_code = e.response['Error']['Code']
            print(f"Attempt {attempt} failed: {error_code}")
            if error_code in ["ThrottlingException",
                "ServiceUnavailableException"]:
                sleep_time = base_delay * (2 ** (attempt - 1)
                sleep_time = sleep_time + random.uniform(0, 1)
                print(f"Retrying in {sleep_time:2f} seconds...")
                time.sleep(sleep_time)
                continue
            else:
                # Non-retryable error
                raise e
        except Exception as e:
            # Other unexpected errors
            raise e
```

4 Identifying Retryable vs Permanent Errors

Retryable (Try Again)

- ThrottlingException
- ServiceUnavailableException
- RequestTimeout
- TooManyRequestsException
- SlowDown (some services)
- Temporary network issues

Permanent (Do Not Retry)

- AccessDeniedException
- ResourceNotFoundException
- ValidationException
- InvalidRegionException
- InvalidParameterException
- IdempotentParameterMismatch

5 Return Useful Error Messages

```
{
  "success": false,
  "error": "Access denied while listing S3 buckets.",
  "error_code": "AccessDeniedException",
  "suggestion": "Check your IAM permissions for
   s3:ListBuckets or use a different AWS account.",
  "retryable": false
}
```

6 Handle Model / Tool Errors in Your Agent

- Catch tool invocation errors from your agent framework.
- Return a clear message to the user.
- Do not expose sensitive stack traces.
- Log full error details for debugging.
- Keep the agent usable even if one tool fails.

7 Example: Handle AI Model Errors (Bedrock)

- ThrottlingException: Reduce request rate or add retry.
- ModelNotReadyException: Model is loading, try again.
- ValidationException: Invalid model ID or request format.
- ServiceUnavailableException: Try again later.
- GuardrailBlockedException: Request blocked by guardrails.

8 Best Practices

- Use bounded retries with exponential backoff.
- Add jitter to avoid synchronized retries.
- Distinguish retryable vs permanent errors.
- Return clear, user-friendly error messages.
- Log errors with context (service, action, request ID).
- Set appropriate timeouts for SDK calls.

9 Milestone

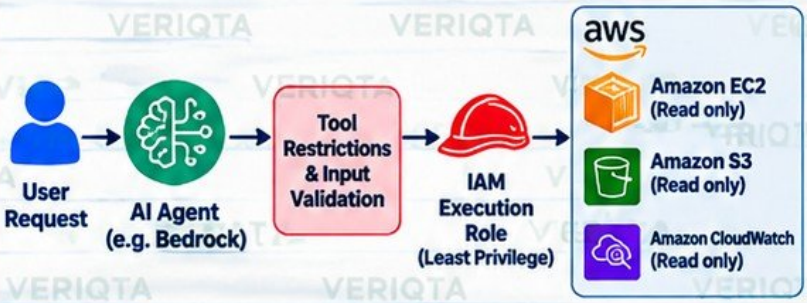
The agent fails safely, reports meaningful errors, and remains usable after a failed request.

Page 15: Secure Your AI Agent with IAM and Tool Restrictions

1. Why Security Matters

- Your AI agent can call AWS services.
- Without proper restrictions, it could accidentally or intentionally change or delete resources.
- Model instructions alone cannot enforce security because the model can be manipulated (prompt injection), make mistakes, or be given unsafe input.
- Use IAM, least privilege and tool restrictions to control what the agent can actually do.

2. Architecture Overview



3. Create a Dedicated IAM Role for the Agent

1. Go to IAM > Roles > Create role.
2. Choose Trusted entity: AWS service (e.g. Amazon Bedrock or Lambda).
3. Attach a custom least-privilege policy (only allow read actions).
4. Give the role a clear name, for example: VeriqtaAIAgentReadOnlyRole.

4. Example Least-Privilege IAM Policy (Read-Only Access)

```
{
   "Version": "2012-10-17",
   "Statement": [
     {
         "Effect": "Allow",
         "Action": [
            "ec2:Describe*",
            "s3:List*",
            "s3:Get*",
            "cloudwatch:Get*",
            "cloudwatch:List*"
         ],
         "Resource": "*"
     }
   }
}
```

What This Prevents

- Creating resources (e.g. ec2:RunInstances)
- Deleting resources
- Modifying resources (e.g. ec2:TerminateInstances)
- Changing IAM settings
- Accessing services not in the policy

Best Practice

- Only allow the specific read actions the agent needs.
- Keep permissions as narrow as possible.
- Use separate roles for different agents or tools.

5. Restrict Tools and Validate Inputs

- Define only the tools the agent can use (e.g. list EC2, read S3 object, view CloudWatch logs).
- Validate all user inputs before executing a tool.
- Reject inputs that try to perform unauthorized actions (e.g. create, delete, modify).
- Use allowlists for parameters (e.g. only specific bucket names or instance IDs).

6. Protect Credentials

- Use IAM roles instead of access keys.
- Do not hardcode credentials in code or prompts.
- Use AWS Secrets Manager if any secrets are required.
- Rotate credentials regularly.

7. Defend Against Prompt Injection

- Do not rely on model instructions for security).
- Validate and sanitize all user inputs.
- Use a fixed system prompt that rejects unauthorized actions.
- Implement tool-level guards and parameter checks.
- Log and monitor all agent actions.

8. Test the Security Controls

- Ask the agent to list EC2 instances (should work).
- Ask the agent to create an EC2 instance (should be denied).
- Check CloudWatch logs for denied actions.
- Verify the agent can only inspect authorized resources.

9. Milestone

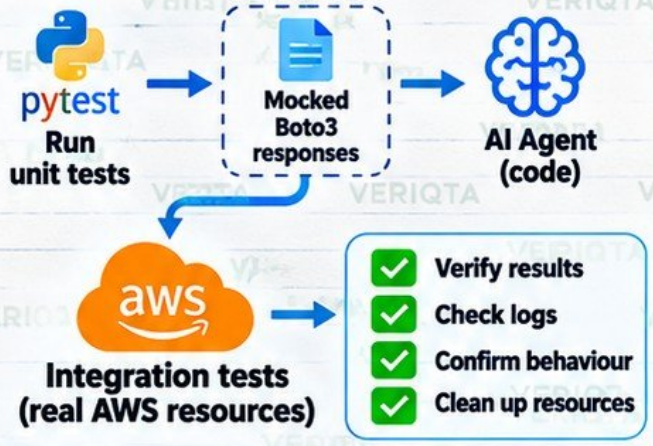
The agent can inspect authorized resources but cannot create, delete, or modify them.

Test the AI Agent with Real AWS Scenarios

1. What We Are Testing

- **Tool selection** (chooses the right tool)
- **Invalid input** (handles bad requests)
- **Empty resource lists** (returns helpful response)
- **Denied permissions** (graceful error handling)
- **Timeout handling** (retries or gives clear message)
- **Unexpected model output** (validates and recovers)
- **Integration with real AWS resources** (end-to-end)

2. Test Architecture



3. Project Structure

```
ai-agent/
├── agent.py          # your AI agent
└── tools.py          # AWS tool functions
tests/
├── test_agent.py     # unit tests
└── test_integration.py  # live AWS tests
requirements.txt
```

4. Example Unit Test (Mocked Boto3)

Tool selection test

```python
import pytest
from unittest.mock import patch, MagicMock
from agent import run_agent

@patch("boto3.client")
def test_list_s3_buckets_success(mock_boto):
    mock_s3 = MagicMock()
    mock_s3.list_buckets.return_value = {
        "Buckets": [{"Name": "demo-bucket"}]
    }
    mock_boto.return_value = mock_s3

    response = run_agent("List my S3 buckets")
    assert "demo-bucket" in response
```

5. Test Invalid Input

Invalid input test

```python
def test_invalid_input():
    response = run_agent("")
    assert "please provide" in response.lower()
```

6. Test Empty Resource List

Empty result test

```python
@patch("boto3.client")
def test_empty_ec2_list(mock_boto):
    mock_ec2 = MagicMock()
    mock_ec2.describe_instances.return_value = {
        "Reservations": []
    }
    mock_boto.return_value = mock_ec2

    response = run_agent("List my EC2 instances")
    assert "no ec2 instances" in response.lower()
```

7. Test Denied Permissions

Permission error test

```python
@patch("boto3.client")
def test_access_denied(mock_boto):
    mock_s3 = MagicMock()
    mock_s3.list_buckets.side_effect = \
        Exception("AccessDenied: User is not authorized")
    mock_boto.return_value = mock_s3

    response = run_agent("List my S3 buckets")
    assert "not authorized" in response.lower()
```

8. Test Timeout Handling

Timeout test

```python
@patch("boto3.client")
def test_timeout(mock_boto):
    mock_s3 = MagicMock()
    mock_s3.list_buckets.side_effect = TimeoutError("Request timed out")

    mock_boto.return_value = mock_s3

    response = run_agent("List my S3 buckets")
    assert "timed out" in response.lower()
```

9. Integration Test (Real AWS)

Live AWS test

```python
def test_real_s3_list():
    response = run_agent("List my S3 buckets in us-east-1")
    assert isinstance(response, str)
    assert len(response) > 0
    assert "bucket" in response.lower()
```

10. Milestone

- **The agent passes repeatable automated tests and a verified live AWS inspection.**

Page 17: Deploy Your AI Agent with AWS Lambda

1 What You Will Do

- Package the Python application and dependencies.
- Create a Lambda execution role.
- Configure environment variables.
- Set memory and timeout limits.
- Deploy the handler.
- Test it with a sample event.
- Use role-based authentication (instead of local AWS credentials).

2 Architecture Overview



3 Package Your Python Application

```
# 1. Create project folder
mkdir ai-agent-lambda
cd ai-agent-lambda

# 2. Create virtual environment
python3 -m venv venv
source venv/bin/activate

# 3. Install required packages
pip install boto3 openai -t .

# 4. Add your code file
cp lambda_function.py .

# 5. Create deployment package
zip -r function.zip .
```

4 Lambda Function Code (lambda_function.py)

```
import json
import boto3
bedrock = boto3.client('bedrock-runtime')

def lambda_handler(event, context):
    user_input = event.get('prompt', 'Hello')

    # Call Amazon Bedrock (example using Claude)
    response = bedrock.invoke_model(
        modelId='anthropic.claude-3-haiku-20240307-v1:0',
        body=json.dumps({
            "messages": [{
                "role": "user",
                "content": [{"text": user_input}]
            }]
        })
    )

    result = json.loads(response['body'].read())
    answer = result['content'][0]['text']

    return {
        "statusCode": 200,
        "body": json.dumps({"response": answer})
    }
```

5 Create a Lambda Execution Role

1. Open IAM → Roles → Create role
2. Select AWS service → Lambda
3. Attach policies:
   - AmazonBedrockFullAccess
   - CloudWatchLogsFullAccess
4. Name the role (e.g. ai-agent-lambda-role)
5. Create the role.

6 Deploy the Lambda Function

1. Open Lambda → Create function
2. Choose Author from scracth
3. Function name: ai-agent
4. Runtime: Python 3.11 (or latest)
5. Upload function.zip
6. Select the execution role
7. Set memory: 512 MB (or higher)
8. Set timeout: 1–2 minutes
9. Add environment variables (if needed)
10. Create function.

7 Test the Function

```
{
  "prompt": "Explain what
  an AI agent is in simple
  terms."
}
```

▶ Test

Check the response in the output. You should see the AI agent's answer.

⚠ Important Configuration

- Use a suitable memory size for your model.
- Set a timeout long enough for the response.
- Use environment variables for model IDs and configuration.
- Never hardcode AWS credentials.

⚙ Common Issues

- AccessDeniedException → check IAM role permissions.
- Task timed out → increase timeout or memory.
- Module not found → include all dependencies in the zip file.
- Model access error → verify Bedrock model access in your AWS account.

✓ Milestone

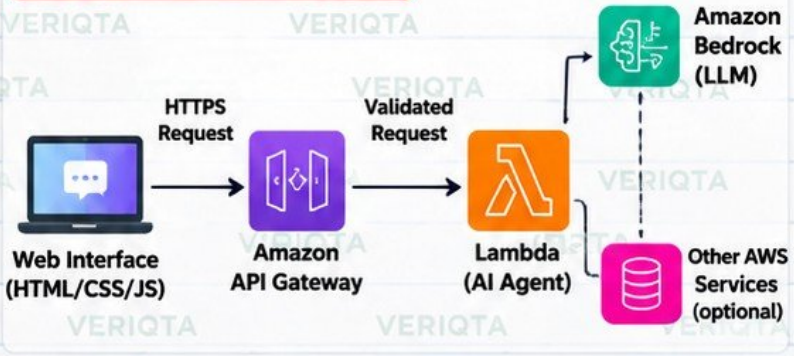
The AI agent runs successfully inside AWS Lambda.

Build a Web Interface and Connect API Gateway

1. What We Are Building

- A simple web chat interface (HTML, CSS, JavaScript).
- Amazon API Gateway to expose your Lambda function.
- Request validation and CORS configuration.
- Authentication to protect the endpoint.
- The browser connects to the AI agent through API Gateway.

2. Architecture Overview



3. Build a Simple Web Chat Interface

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>VERIQTA AI Agent</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h2>VERIQTA AI Agent</h2>
    <div id="chat-box"></div>
    <div class="input-area">
      <input type="text" id="message"
        placeholder="Type your message...">
      <button onclick="sendMessage()">Send</button>
    </div>
  </div>
  <script src="app.js"></script>
</body>
</html>
```

style.css

```
body { font-family: Arial; background: #f5f5f5; }
.container { max-width: 800px; margin: 40px auto;
  background: #fff; padding: 20px; border-radius: 8px;
  box-shadow: 0 2px 8px #ddd; }
#chat-box { height: 400px; overflow-y: auto;
  border: 1px solid #ddd; padding: 10px; margin-bottom: 10px;}
.input-area { display: flex; }
#message { flex: 1; padding: 10px; font-size: 16px; }
button { padding: 10px 20px; background: #00382F;
  color: #fff; border: none; font-size: 16px; }
```

app.js

```
async function sendMessage() {
  const message = document.getElementById("message").value;
  if (!message) return;
  const chatBox = document.getElementById("chat-box");
  chatBox.innerHTML += `<p><b>You:</b> ${message}</p>`;
  const response = await fetch("https://YOUR_API_ID.execute-api.
    "us-east-1.amazonaws.com/prod/chat", {
      method: "POST",
      headers: { "Content-Type": "application/json",
                 "Authorization": "Bearer YOUR_TOKEN" },
      body: JSON.stringify({ message: message })
  });
  const data = await response.json();
  chatBox.innerHTML += `<p><b>Agent:</b> ${data.reply}</p>`;
  document.getElementById("message").value = "";
}
```

4. Create and Configure API Gateway

1. Create a REST or HTTP API in API Gateway.
2. Create a POST /chat route.
3. Integrate with your Lambda function.
4. Enable request validation (JSON schema).
5. Enable CORS to allow your web interface.
6. Deploy to a stage (e.g., prod).
7. Note the endpoint URL.

5. Enable CORS (Example)

```
{
  "Access-Control-Allow-Origin": "https://yourdomain.com",
  "Access-Control-Allow-Methods": "POST,OPTIONS",
  "Access-Control-Allow-Headers": "Content-Type,Authorication"
}
```

6. Add Authentication (Example)

- Use Amazon Cognito (recommended) or an API key.
- In the browser, get a valid token after login.
- Include the token in the Authorization header.
- Only authenticated users can access the AI agent.

7. Test in Your Browser

- Open index.html in a browser.
- Log in and get a valid token.
- Send a message to the agent.
- Verify the response is returned.
- Check API Gateway logs if there are errors.

8. Milestone

An authenticated user can chat with the deployed agent through a browser.

19. Monitor Your AI Agent and Control AWS Costs

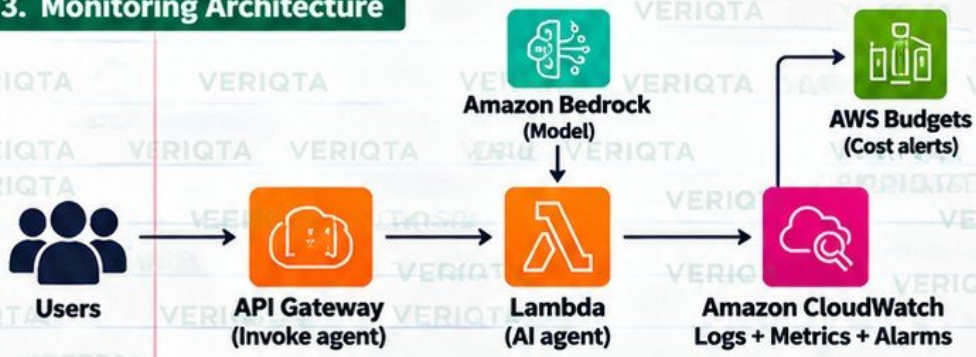
1. Why Monitoring Is Important

- Helps you know if your AI agent is working.
- Tracks performance, errors and costs.
- Helps you detect issues before users report them.
- Prevents unexpected AWS bills.
- Gives you data to improve your agent.

2. What We Will Set Up

1. CloudWatch logs for application and agent.
2. CloudWatch metrics (requests, latency, errors).
3. Bedrock token usage tracking.
4. CloudWatch alarms for failures.
5. AWS Budget alerts for cost control.
6. Test monitoring with a controlled failure.

3. Monitoring Architecture



What We Monitor

- Request count
- Latency (response time)
- Errors and failures
- Logs (CloudWatch Logs)
- Bedrock token usage
- AWS costs (Budgets)

4. Enable CloudWatch Logging (Lambda Example)

```python
import json
import logging

# Configure logging
logger = logging.getLogger()
logger.setLevel(logging.INFO)

def lambda_handler(event, context):
    logger.info(f"Request received: {json.dumps(event)}")

    # Your AI agent code here
    return {"statusCode": 200, "body": "Success"}
```

What This Does

- Sends logs to CloudWatch.
- Helps you trace each request.
- Use structured logs.
- Never log sensitive data like API keys or full user prompts.

Security Tip

Do NOT log credentials, personal data or sensitive prompts.

5. Create CloudWatch Metrics (Custom)

```python
import boto3
cloudwatch = boto3.client('cloudwatch')

cloudwatch.put_metric_data(
    Namespace='AI-Agent',
    MetricData=[{
        'MetricName': 'RequestCount',
        'Value': 1,
        'Unit': 'Count'
    }]
)
```

Common Metrics

- RequestCount – number of requests
- Latency – response time
- ErrorCount – failed requests
- TokenUsage – tokens used from Bedrock

6. Create CloudWatch Alarms

Create alarm

Metric: AI-Agent-ErrorCount

Condition: Greater than 5

Evaluation period: 1 minute

Create alarm

Example Alarms

- ErrorCount > 5 (1 minute)
- Latency > 5 seconds
- Lambda errors (built-in metric)
- API Gateway 5XX errors

Notify You

- Email (SNS)
- SMS (optional)
- Slack (via SNS)

7. AWS Budgets for Cost Control

Create budget

Budget type: Cost budget

Budget amount: 20 USD

Alert threshold: 80% 100%

Create budget

Best Practices

- Set a low budget while testing.
- Enable 80% and 100% alerts.
- Monitor Bedrock model usage.
- Review costs regularly.
- Delete unused resources.
- Use the AWS Free Tier where possible.

8. Test Monitoring with a Controlled Failure

1. Temporarily use an invalid model ID in your code.
2. Send a test request to your agent.
3. Check CloudWatch logs for the error.
4. Verify the CloudWatch alarm triggers.
5. Confirm you receive the SNS notification.

Milestone

- You can view logs for each request.
- You can see key metrics (requests, latency, errors).
- You can track Bedrock token usage and AWS costs.
- You receive alerts when something fails.
- You can investigate failures using CloudWatch.
- You have cost controls in place with AWS Budgets.

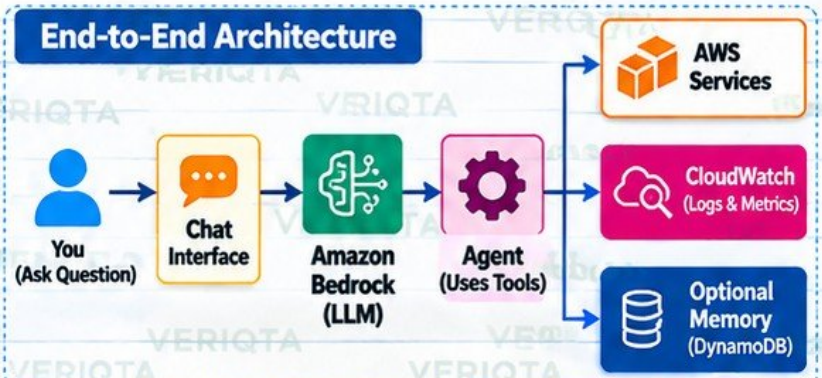
Your First AI Agent on AWS:
Final Project, End-to-End Verification, and Cleanup

1 What You Built

An AI Cloud Operations Assistant that:

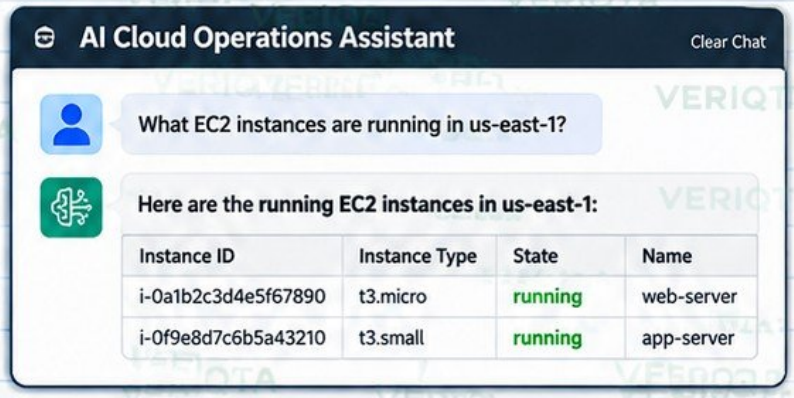
- Understands natural language questions
- Uses AWS services securely
- Runs with IAM permissions (least privilege)
- Returns real-time information from AWS
- Maintains conversation memory (optional)
- Is monitored and can be safely cleaned up

End-to-End Architecture



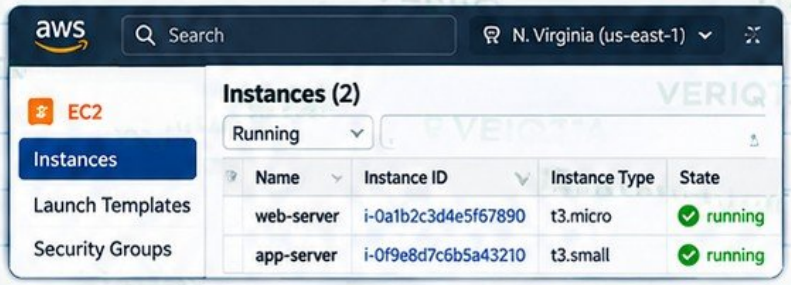
2 Test the Complete Agent

1. Open your chat interface (web UI or CLI).
2. Ask real AWS questions.
3. Verify the information in the AWS console.
4. Test different types of questions.
5. Check conversation memory (if implemented).
6. Try allowed and disallowed actions.
7. Monitor logs and metrics.

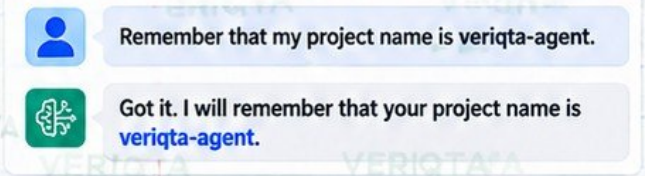


3 Verify in AWS Console

- Log in to the AWS Management Console.
- Go to the service (e.g., EC2, S3, RDS).
- Confirm the agent's response matches the console.
- Check that no incorrect information is returned.



4 Test Conversation Memory (Optional)



5 Simulate Failures

- Ask a question outside the allowed IAM permissions.
- Example: "Delete this S3 bucket" (should be denied).
- Verify the agent handles the error correctly.
- Check CloudWatch logs for the failure.
- Confirm no unintended changes were made.

6 Check Monitoring

- Open Amazon CloudWatch.
- View logs for agent requests and responses.
- Check metrics (invocations, errors, latency).
- Confirm alarms if configured.

7 Cleanup Resources

- Stop or delete compute resources (EC2, ECS, Lambda).
- Delete test S3 buckets and data.
- Remove DynamoDB tables (if used).
- Delete IAM policies, roles, and users.
- Remove CloudWatch log groups (optional).
- Delete other resources to avoid charges.

8 Milestone

A working, secured, monitored AI agent has been demonstrated and its cleanup verified.